Derivation of Automatic Test Set for Detection of Missing Gate Faults in Reversible Circuits

Dipak K. Kole[1], Hafizur Rahaman[1], Debesh K Das[2], and Bhargab B. Bhattacharya[3]
[1]Information Technology Department, Bengal Engineering & Science University, Shibpur, Howrah, India
{*dipak.kole@gmail.com, rahaman_h@it.becs.ac.in*}
[2]Dept. of Computer Sc. and Engg., Jadavpur University, Jadavpur, India {*debeshd@hotmail.com*}
[3]ACM Unit, Indian Statistical Institute, Kolkata, India {*bhargab@isical.ac.in*}

***Abstract*—**This article presents a novel technique for the generation of test set in a reversible quantum circuit. The algorithms are developed to derive the automatic test set (ATS) for the detection of all partial missing-gate faults, all single missing gate faults and multiple missing gate faults in an *(n x n)* reversible circuit implemented with *k*-CNOT gates. Experimental results on some benchmark circuits are also reported.

Keywords — Missing-gate faults, quantum computing, reversible logic, testable design, universal test set.

1. INTRODUCTION

Reversible logic can be employed to design information lossless circuits, which may reduce power consumption drastically [1-2]. It also provides a basis for the newly emerging paradigm of quantum computing [3]. Reversible logic will offer a viable solution in the future with newer technologies, e.g., spintronics. Reversible circuits have numerous applications to optical computing, cryptography, nanotechnology, DNA technology, and low-power CMOS design [3-5].

An *n*-input, *m*-output Boolean function *F* is said to be reversible if and only if $m = n$, and *F* is one-to-one. A reversible combinational circuit should consist of only reversible gates, which themselves implement reversible functions; such gates need to be specially designed, e.g., by using controlled-not (CNOT), Feynman [6], Fredkin [7] or Toffoli [8] gates.

Recently, several researchers have studied the problem of fault modeling and testing of reversible logic circuits [16-22]. Universal testability of reversible logic circuits implemented with *k*-CNOT gates under the stuck-at fault model has been investigated [20]. However, new fault models, namely, SMGF, RGF, PMGF, and MMGF, have been found to be more suitable for modeling defects in quantum *k*-CNOT gates [17, 22]. A *k*-CNOT based circuit can be implemented using trapped-ion technology, where the ions interact with laser pulses [3, 22]. Determination of a universal test set for fault detection of the faults in reversible circuits has been studied [17, 20]. The problem of detecting MMGFs has been investigated [22]. The technique in [14] admits a universal test set of size $(n + 1)$ that detects all SMGFs, RGFs, and PMGFs in an $(n \times n)$ reversible circuit implemented with *k*-CNOT gates. In this work, we have developed efficient algorithms to derive an automatic test set (ATS) for the detection of all partial missing-gate faults, all single missing gate faults and multiple missing gate faults in an *(n x n)* reversible circuit implemented with *k*-CNOT gates.

This paper is organized as follows. In Section 2, some preliminaries of reversible logic circuits are presented. Then, Section 3 discuses fault detection in a reversible logic circuit. Experimental results are shown in Section 4. Finally, Section 5 concludes the work.

This work is partially supported by SMDP-II, sponsored by DIT, MCIT, Govt. of India.

2. PRELIMINARIES

Reversible Logic: A reversible function simply induces a permutation on the set of input vectors to produce an output vector. Therefore, given an output vector, its corresponding input vector can be uniquely restored. A non-reversible Boolean function can always be implemented by a reversible circuit after appropriately transforming it to a reversible one by adding garbage lines and reversible gates [14]. By *garbage* is meant the number of extra outputs required to realize the given function.

Reversible gates: The basic CNOT type reversible gates used for synthesis are the following (Fig. 1):

(i) 1-input/1-output NOT ($x_1 \rightarrow \overline{x_1}$);
(ii) 2-input/2-output controlled NOT (CNOT) gate: $(x_1, x_2) \rightarrow (x_1, x_1 \oplus x_2)$;
and (iii) 3-input/3-output Toffoli gate $(x_1, x_2, x_3) \rightarrow (x_1, x_2, x_1 x_2 \oplus x_3)$.

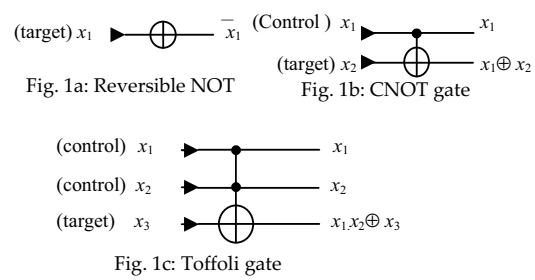

Fig. 1a: Reversible NOT
Fig. 1b: CNOT gate
Fig. 1c: Toffoli gate

A *k*-CNOT gate has *k* control inputs $x_1, x_2, \ldots, x_k$ and one target input *t*. It maps the input vector $(x_1, x_2, \ldots, x_k, t)$ to the output vector $(x_1, x_2, \ldots, x_k, t \oplus x_1 . x_2 \ldots x_k)$. Thus, a *k*-CNOT gate has $k+1$ inputs and $k+1$ outputs; the first *k* outputs follow the respective inputs, and it inverts the target at the $(k+1)^{th}$ output if and only if all the *k* control inputs are 1.

Fault model*:* Several new fault models for *k*-CNOT based reversible circuits have been introduced [17, 22]. These are single missing-gate fault (SMGF), the repeated gate fault (RGF), partial missing-gate fault (PMGF), and the multiple missing-gate fault (MMGF). In this section, we briefly explain with examples, the nature of these faults.

Single missing-gate fault (SMGF): This model corresponds to

978-0-7695-4570-7/11 $26.00 © 2011 IEEE
DOI 10.1109/ISED.2011.69

IEEE computer society

the case when one k-CNOT gate completely disappears from the circuit. In the presence of this fault, the CNOT gate behaves as a simple wire connection, i.e., the pulse implementing the gate operation is short, missing, misaligned or mistuned.

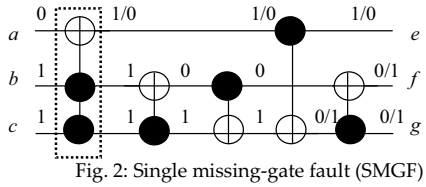

Fig. 2: Single missing-gate fault (SMGF)

Fig. 2 shows an SMGF marked by the dotted box in the reversible circuit ham3\design#1 benchmark, where the first 2-CNOT gate is missing. An SMGF is detected by setting logic 1 value on all the control inputs of the gate, and any value either 0 or 1 on the target input as well as on the wires not connected to the gate. In the example of Fig. 2, if we apply $\{a, b, c\} = \{0, 1, 1\}$ at the input of the circuit, the normal output would be $\{e, f, g\} = \{1, 0, 0\}$, whereas, in the presence of the SMGF fault marked by the dotted box, the output will be $\{e, f, g\} = \{0, 1, 1\}$. The number of possible *SMGFs* is equal to the number of gates in the circuit.

Repeated-gate fault (RGF): A repeated-gate fault *(RGF)* is an unwanted replacement of a k-CNOT gate by several instances of the same gate [22]. An RGF may be needed to model the occurrence of long or duplicated pulses. Fig. 3 shows an example, where the first gate is repeated in the circuit ham3\design#1. The effect of this fault is thus same as that of an SMGF at the first 2-CNOT gate in the original circuit. If we apply $\{a, b, c\} = \{0, 1, 1\}$, the normal output would be $\{e, f, g\} = \{1, 0, 0\}$, whereas, in the presence of the above RGF marked by the dotted box, the output will be $\{e, f, g\} = \{0, 1, 1\}$. Hence, it is detected by the vector $\{a, b, c\} = \{0, 1, 1\}$.

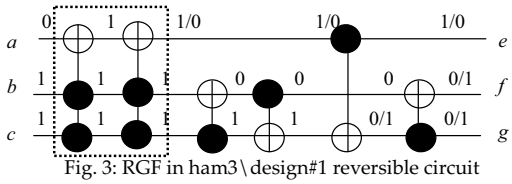

Fig. 3: RGF in ham3\design#1 reversible circuit

It is clear that if a RGF replaces a gate by even number of instances of the same gate, its effect is similar to the effect of the SMGF with respect to the same gate. If the RGF replaces a gate by odd number of instances of the same gate, the fault is redundant, i.e., it does not change the function of the circuit. Further, it has been shown that any SMGF test set detects all detectable RGFs [22].

Partial missing-gate fault (PMGF): This is used to model the defects resulting from the partially misaligned or mistuned gate pulses [22]. It changes a k-CNOT gate into a p-CNOT gate, with $p < k$. The corresponding fault is called as $(k - p)^{th}$ *order* PMGF. Fig. 4 shows a first-order PMGF affecting the second control input of the leftmost gate. An SMGF can be seen as a 0-order PMGF. In the circuit of Fig. 4, if we apply $\{a, b, c\} = \{1, 0, 1\}$, the normal output would be $\{e, f, g\} = \{1, 1, 0\}$, whereas, in the presence of the first order PMGF fault as shown, the output will be $\{e, f, g\} = \{0, 0, 1\}$. Hence, the vector $\{a, b, c\} = \{1, 0, 1\}$ detects this fault. It has been shown [22] that a PMGF (of first or higher order) is detected when a 0 is applied to at least one of the affected control inputs and a 1 to all other control inputs. Thus, a higher order PMGF is detected by a test vector for a first order PMGF, the affected control input of which is one of those affected in the higher order PMGF. Hence, it is sufficient to consider first order PMGFs only.

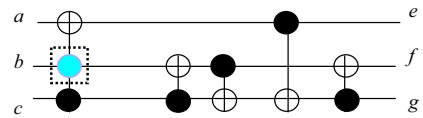

Fig. 4: PMGF in ham3\design#1

Multiple missing-gate fault (MMGF): This is defined as complete disappearance of two or more *consecutive* k-CNOT gates from the circuit.

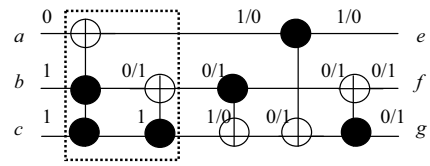

Fig. 5: MMGF in *ham3\design#1*

In the circuit of Fig. 5, it is shown that the circuit has an MMGF marked by the dotted box. This fault is detected by the vector $\{a, b, c\} = \{0, 1, 1\}$.

3. FAULT DETECTION IN A REVERSIBLE CIRCUIT

In this paper, we propose algorithms for automatic test set (ATS) construction for detecting all PMGFs, SMGFs, and MMGFs respectively. The automatic test set corresponding to each type missing gate fault is constructed using a bipartite graph modelling.

A. Generation of Permutation

Each Toffoli gate with n inputs induces a permutation on the set $\{0, 1, 2, \ldots, 2^n-1\}$. In this paper, considering Toffoli gates of size n means generation of permutations of size n. The permutation of corresponding to each gate is stored in an array and the array index is considered as the gate *Id*. The input lines are numbered as $0,1,2,\ldots,(n-1)$ starting from the top. The selection of target line *tl* is made from line 0 to line $(n-1)$ one by one. After selecting the target line *tl*, the next task is to select the set of control lines from all possible sets of control lines with respect to the target line number *tl*. In other words, a total of 2^{n-1} sets of control lines with respect to the target line can be generated. We can use a binary number $b_{n-2}\, b_{n-3} \ldots b_1\, b_0$ to represent the corresponding control set. In other words, if a bit $b_i = 0$, then the corresponding line is not considered as a control line, whereas, if $b_j = 1$, then the corresponding line is to be added as a control line of that control set.

We next apply all possible test patterns $\{0,1,2,\ldots\ldots,2^n-1\}$ and collect the corresponding set of output test pattern which can be represented as a permutation:

$$G_i = \begin{pmatrix} 0 & 1 & 2 & \cdots & (2^n-1) \\ G_i(0) & G_i(1) & G_i(2) & & G_i(2^n-1) \end{pmatrix} = (G_i(0)\, G_i(1) \ldots G_i(2^n-1))$$

where $G_i(t)$ is the output test pattern of the input test pattern t and i is the identity number of that gate. In this way, all permutations for a Toffoli gates of size n are generated and stored in an array. Table 1 lists all Toffoli gates of size n=3

with gate *Id* and the corresponding permutation.

Table 1: Gate *Id*s and corresponding permutations of all Toffoli gate of size 3

Gate Ids	Corresponding Permutation
0	(4, 5, 6, 7, 0, 1, 2, 3)
1	(0, 5, 2, 7, 4, 1, 6, 3)
2	(0, 1, 6, 7, 4, 5, 2, 3)
3	(0, 1, 2, 7, 4, 5, 6, 3)
4	(2, 3, 0, 1, 6, 7, 4, 5)
5	(0, 3, 2, 1, 4, 7, 6, 5)
6	(0, 1, 2, 3, 6, 7, 4, 5)
7	(0, 1, 2, 3, 4, 7, 6, 5)
8	(1, 0, 3, 2, 5, 4, 7, 6)
9	(0, 1, 3, 2, 4, 5, 7, 6)
10	(0, 1, 2, 3, 5, 4, 7, 6)
11	(0, 1, 2, 3, 4, 5, 7, 6)

B. Automatic Test Sets for Detecting Missing-Gate Faults

Let us consider a reversible circuit *RC* of size *n* with depth *d* consisting of $\{g_0.g_1.g_2...g_{d-1})$ reversible gates. Compute the reversible specification *f* of the circuit RC using $f = P(g_0).P(g_1)\cdots P(g_{N-1})$ and its inverse f^{-1}.

Let B_i^{pre} denote the predecessor block of the gate g_i, consisting of a sequence of gates from level 0 to level *(i-1)* of the circuit RC,

i.e., $B_i^{pre} = \{g_0, g_1, \cdots, g_{i-1}\}\ \forall\, 0 < i \le N-1$.

Let B_j^{post} denote the post block of the gate g_j, consisting of a sequence of gates from level (*j*+1) to level (*d*-1) of the circuit RC, i.e., $B_j^{post} = \{g_{j+1}, g_{j+2}, \cdots, g_{N-1}\}\ \forall\, 0 \le j < N-1$.

Let B_{ij} denote a block consisting of a sequence of gates from level *i* to level *j* of the circuit RC, i.e. $B_{ij}=\{g_i, g_{i+1}, \ldots, g_j\}$ for all $0 \le i \le j \le (N-1)$.

Since there is no gate before the first gate of the circuit and after the last gate of the circuit, we have,

for $i = 0, B_0^{pre} = \{\} = \phi$ and $j = N-1, B_{N-1}^{post} = \{\} = \phi$.

Let $P(B_{ij})$ denote the permutation of the block B_{ij} which is a product of $P(g_i), P(g_{i+1}), \ldots, P(g_j)$ where $P(g_k)$ denotes the permutation of the gate g_k for all $k = 0$ to $(N-1)$.

This implies that these two blocks contain no gate. If we apply any test pattern *t* as an input to these blocks, we will get the same output test pattern *t*. Hence, $P(B_0^{pre}) = P(B_{N-1}^{post}) = I$, where I is the identity permutation on the set $\{0,1,\cdots,2^n-1\}$.

Automatic Test Set Construction for PMGFs

Let us assume that T_i denotes a set of faults (fault numbers) which are detected by the test pattern i, where $0 \le i < (2^n-1)$ and F_j denotes a set of test patterns which detect the fault j (fault level *j*), where $0 \le j <$ (total number of controls of the circuit RC). Initially all T_i and F_j sets are empty, this means $T_i=\{\ \}$ and $F_j=\{\ \}$ where *i* varies from 0 to (2^n-1) and *j* varies from 0 to (total number of controls of the circuit RC).

Initially, ATS={ }.

Let us consider the partial missing gate fault number *j,* which appears at the ith level (i.e. g_i gate) of the circuit RC.

To detect the fault *j* (only a single control is missing from the gate *gi*), we have to supply those test patterns at that level *i*, which contain a zero corresponding to the missing control position and all ones corresponding to all remaining control positions of the gate g_i.

Let *t* be a test pattern which satisfies the above criteria.

Now, we have to find the test pattern *p* at the input level of the circuit RC, which can produce the test pattern *t* at the ith level of the circuit RC.

We compute the permutation,

$P(B_i^{pre}) = p(g_0).p(g_1)..p(g_{i-1}) = f_i$ (say)

Next, we compute the inverse permutation f_i^{-1}.

So, the test pattern *p* can be computed as follows:

$p=f_i^{-1}(t)$.

Next, we add *p* (test pattern) and *j* (fault number) elements to the sets F_j and T_p respectively.

We continue for all faults and construct the sets T_i and F_j completely where *i* varies from 0 to (2^n-1) and *j* varies from 0 to (total number of controls of the circuit RC).

After the construction of the sets T_i and F_j, we construct the bipartite between the set of all test patters and the set of all PMGF of order 1. Then find out the minimal degree based on the above observations. We describe the following algorithm that generates an automatic test set (*ATS*) for detecting all possible MMGFs in a reversible circuit.

Algorithm_PMGF:

Input: A reversible circuit RC of size n with depth N consisting N number of gates g0, g1,…., gN-1 .

Output: Automatic Test set (ATS) for detecting all possible multiple missing-gate faults of the circuit RC.

Step1: Consider a record Set consist of two fields:

Record Set:

1) noe //denotes the total number of elements
2) elements[] //stores the members of the set

Step2: fault_level=0 , total_fault=0 and ATS={}// Initialize fault_level, total_fault by zeros and Automatic Test Set by NULL.

Consider the two one-dimensional arrays test_set[] and fault_set[] of Set type where the row number of test_set[] array represent test vector in decimal form and the set corresponding to the row represent a set of fault numbers detected by the corresponding row test vector. The row number of fault_set[] array represent fault number and the set corresponding to the row represent set of test vectors which detect the fault represented by the row number.

Step3: For glevel=0 to (*N*-1)

1.1 f_{glevel}=compute_permutation_product(glevel)

/*this function perform permutation product of glevel number of gates starting from first gate of the circuit RC. If the value of glevel is zero then this function return identity permutation*/

1.2 f_{glevel}^{-1}=compute_inverse_permutation(f_{glevel})

1.3 compute_gate_details(glevel, nc, gate_details[])

/*This function compute the total number of controls and their corresponding control positions of the gate present at the level glevel of the circuit RC and here nc store the total no. of control and position of the controls are stored in the gate_details array data structure*/

1.4 For control=1 to *nc*

1.4.1 For *t*=0 to (2^n-1) // *t* denotes the test pattern in decimal form

1.4.1.1 Check whether the test pattern *t* contains a zero value at the position of gate_details[control] and all ones in remaining all control positions or not.

1.4.1.2 If the test vector *t* satisfy the above condition

Then find out p=f_{glevel}^{-1}(t) and n_1=test_set[p].noe , test_set[p].elements[n1]=fault_level, n_2=fault_set[fault_level].noe, fault_set[fault_level].elements[n_2]=p, test_set[p].noe=test_set[p].noe+1 fault_set[fault_level].noe= fault_set[fault_level].noe+1

1.4.2 fault_level= fault_level+1;

Step4: Construct the Bipartite graph using two arrays test_set[] and fault_set[] array

Step5: Find the fault number f_{min} containing minimum degree of fault level from the fault_set array and then find out maximal degree test vector t_{max} from set of test patterns contained in fault_set [f_{min}]

Step6: ATS=ATS U { t_{max} }

Step7: Delete t_{max} from test_set[] array and delete all fault numbers from fault_set[] array those are detected by the test vector t_{max}

Step8: Continue the steps from step4 to step7 until fault_set[] is empty.

Automatic Test Set Construction for SMGFs

Let us assume, T_i denotes a set of faults (fault numbers) which are detected by the test pattern i, where 0≤ i<(2^n-1) and F_j denotes a set of test patterns which detect the fault j (fault level j), where 0≤ j <(total number of controls of the circuit RC).

Initially all T_i and F_j sets are empty that means T_i={ } and F_j={ } where *i* varies from 0 to (2^n-1) and *j* varies from 0 to (total number of controls of the circuit RC). Initially, ATS={ }. Let us consider the single missing gate fault appears at the ith level (i.e. g_i gate is missing) of the circuit RC.

We compute the permutations,

$P(B_i^{pre}) = p(g_0).p(g_1)..p(g_{i-1})$,

$P(B_i^{post}) = p(g_{i+1}).p(g_{i+2})..p(g_{N-1})$,

$f_i = P(B_i^{pre}).P(B_i^{post})$ and $P_i = f_i.f^{-1}$.

Decompose the final permutation P_i into disjoint *k* (say) cycles as given below:

$$P_i = (n_1^1 n_2^1 \ldots n_{r1}^1)\cdot(n_1^2 n_2^2 \ldots n_{r2}^2)\ldots(n_1^k n_2^k \ldots n_{rk}^k),$$

where n_j^i is the jth element of the ith cycle.

$$F_i = \{n_1^1, n_2^1, \ldots, n_{r1}^1, n_1^2, n_2^2, \ldots, n_{r2}^2, \ldots, n_1^k, n_2^k \ldots, n_{rk}^k\}$$

We add fault number *i* to all $T_{n_j^i}$ sets.

We continue for all faults and construct the sets T_i and F_j completely where *i* varies from 0 to (2^n-1) and *j* varies from 0 to (total number of gates of the circuit RC).

After the construction of the sets T_i and F_j , we construct the bipartite between the set of all test patters and the set of all SMGFs. Then we apply the same method what we have done in case PMGFs.

Algorithm_SMGF:

Input: A reversible circuit RC of size n with depth *N* consisting of *k*-CNOT gates $g_0, g_{1,\ldots,} g_{N-1}$.

Output: Automatic Test set (ATS) for detecting all possible single missing-gate faults of the circuit RC.

Step 1: Compute the reversible specification f of the circuit RC using $f = P(g_0).P(g_1)\cdots P(g_{N-1})$ and its inverse f^{-1}.

Step 2: for i = 0 to (N-1)

2.1. Let us assume that a single missing-gate fault has occurred in the circuit RC at i^{th} level.

2.2. Compute the permutations $P(B_i^{pre})$ and $P(B_i^{post})$.

2.3. Compute the permutation $f_i = P(B_i^{pre}).P(B_j^{post})$

2.4. Compute the final permutation corresponding to the missing gate g_i as $P_i = f_i.f^{-1}$

2.5. Decompose the final permutation P_i into disjoint k (say) cycles as given below:

$$P_i = (n_1^1 n_2^1 \ldots n_{r1}^1)\cdot(n_1^2 n_2^2 \ldots n_{r2}^2)\ldots(n_1^k n_2^k \ldots n_{rk}^k)$$

, where n_j^i is the jth element of the ith cycle.

2.6. Construct a set $F_i = \{n_1^1, n_2^1, \ldots, n_{r1}^1, n_1^2, n_2^2, \ldots, n_{r2}^2, \ldots, n_1^k, n_2^k \ldots, n_{rk}^k\}$ from the k disjoint cycle elements.

2.7. Add fault number i to all $T_{n_j^i}$ sets.

Step 3: Construct the Bipartite graph using two arrays test_set[] and fault_set[] arrays.

Step 4: Find the fault number f_{min} containing minimum degree of fault level from the fault_set array and then find out maximal degree test vector t_{max} from set of test patterns contained in fault_set[f_{min}]

Step 5: ATS=ATS U { t_{max} }

Step 6: Delete t_{max} from test_set[] array and delete all fault numbers from fault_set[] array those are detected by the test vector t_{max}

Step 7: Continue the steps from step4 to step7 until fault_set[] is empty.

Automatic Test Set Construction for MMGFs

The total number of multiple missing-gate faults in an *n*-input reversible circuit with depth *N* is *{N.(N+1)/2}*. These *{N(N+1)/2}* blocks of consecutive gates of the circuit RC may be missing from the circuit RC under multiple missing-gate fault, and are captured by B_{ij} where $0\le i \le j \le N-1$. This implies that *{N.(N+1)/2}* B_{ij} blocks correspond to all possible multiple missing-gate faults of the circuit *RC*. Since the circuit *RC* is fault free, i.e., no one gate is missing from any block B_{ij} for all $0\le i \le j \le N-1$, we have $B_{ij}= g_i .g_{i+1}.\ldots .g_j$. Then we can write, $P(B_{ij})=P(g_i .g_{i+1}.\ldots .g_j)$ and $P(B_{ij})=P(g_i)P(g_{i+1})\ldots P(g_j)$.

Let us assume, T_i denotes a set of faults (fault numbers) which are detected by the test pattern i, where 0<= i <(2^n-1) and F_j denotes a set of test patterns which detect the fault j (fault level j), where 0≤ *j* < (total number of MMGFs of the circuit RC).

Initially all T_i and F_j sets are empty that means T_i={ } and F_j={ } where *i* varies from 0 to (2^n-1) and *j* varies from 0 to *(N*(N+1))*.

Initially, ATS={ }.

Let us assume that a multiple missing-gate fault has occurred in the circuit RC between the level *i* to level *j* of the circuit RC; the B_{ij} block is missing from the circuit.

We compute the permutations,

$P(B_i^{pre}) = p(g_0).p(g_1)..p(g_{i-1})$,

$P(B_j^{post}) = p(g_{j+1}).p(g_{j+2})..p(g_{N-1})$

$f_{ij} = P(B_i^{pre}).P(B_j^{post})$ and $P_{ij} = f_{ij}.f^{-1}$.

Decompose the final permutation P_{ij} into disjoint *k* (say) cycles as given below:

$$P_{ij} = (n_1^1 n_2^1 \ldots n_{r1}^1)\cdot(n_1^2 n_2^2 \ldots n_{r2}^2)\ldots(n_1^k n_2^k \ldots n_{rk}^k),$$

where n_j^i is the jth element of the ith cycle.

$$F_{ij} = \{n_1^1, n_2^1, \ldots, n_{r1}^1, n_1^2, n_2^2, \ldots, n_{r2}^2, \ldots, n_1^k, n_2^k \ldots, n_{rk}^k\}$$

We add fault number *ij* to all $T_{n_j^i}$ sets.

We continue for all faults and construct the sets T_i and F_j completely where *i* varies from 0 to (2^n-1) and *j* varies from 0 to (total number of gates of the circuit RC).

After the construction of the sets T_i and F_j, we construct a bipartite between the set of all test patterns and the set of all

MMGFs. Then we apply the same method what we have done in case PMGFs.

Algorithm_MMGF:

Input: A reversible circuit RC of size n with depth N consisting of k-CNOT gates $g_0, g_{1,\ldots,} g_{N-1}$.

Output: Automatic Test set (ATS) for detecting all multiple missing-gate faults of the circuit RC.

Step 1: Compute the reversible specification f of the circuit RC using $f = P(g_0).P(g_1)\cdots P(g_{N-1})$ and its inverse f^{-1}.

Step 2: for start_level = 0 to (N-1)

2.1. For end_level=start_level to (N-1)

2.1.1. Let us assume that a multiple missing-gate fault has occurred in the circuit RC between the start_level to end_level of the circuit RC; the B_{ij} block is missing from the circuit where i=start_level and j=end_level.

2.1.2. Compute the permutations $P(B_i^{pre})$ and $P(B_j^{post})$.

2.1.3. Compute the permutation f_{ij}= $P(B_i^{pre}) . P(B_j^{post})$, since

$P(B_{ij}^{f}) = I$

2..1.4. Compute the final permutation corresponding to the block Bij as $P_{ij} = f_{ij} . f^{-1}$.

2..1.5. Decompose the final permutation P_{ij} into disjoint k (say) cycles as given below:

$P_{ij} = (n_1^1 n_2^1 \ldots n_{r1}^1)\cdot(n_1^2 n_2^2 \ldots n_{r2}^2)\ldots(n_1^k n_2^k \ldots n_{rk}^k)$, where n_j^i is the jth element of the ith cycle.

2.1.6. Construct a set

$F_{ij} = \{n_1^1, n_2^1, \ldots, n_{r1}^1, n_1^2, n_2^2, \ldots, n_{r2}^2, \ldots, n_1^k, n_2^k \ldots, n_{rk}^k\}$

from the k disjoint cycle elements corresponding to the block B_{ij}.

2.1.7. Add fault number ij to all $T_{n_j^i}$ sets.

Step3: Construct the Bipartite graph using two arrays test_set[] and fault_set[] arrays.

Step4: Find the fault number f_{min} containing minimum degree of fault level from the fault_set array and then find out maximal degree test vector t_{max} from set of test patterns contained in fault_set[f_{min}]

Step5: ATS=ATS U { t_{max} }

Step 6: Delete t_{max} from test_set[] array and delete all fault numbers from fault_set[] array those are detected by the test vector t_{max}

Step7: Continue the steps from step4 to step7 until fault_set[] is empty.

Example3: Consider a *ham3tc* reversible benchmark circuit.

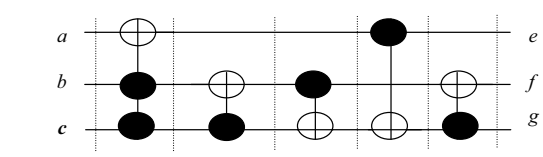

Fig. 6: ham3\design#1 benchmark reversible circuit

In this circuit, the total number of control nodes is six. Therefore, total number of PMGF of order 1 is 6 (fault levels are from 0 to 5). Since, the circuit has three input lines, the total number of test vectors applicable to this circuit is 8 (starting from 0 to 7). First, we construct a bipartite graph between the set of test vectors and the set of all PMGFs of order 1, which is shown in the figure 7 (a). Then, we find a PMGF fault level number from the set of faults having minimum degree.

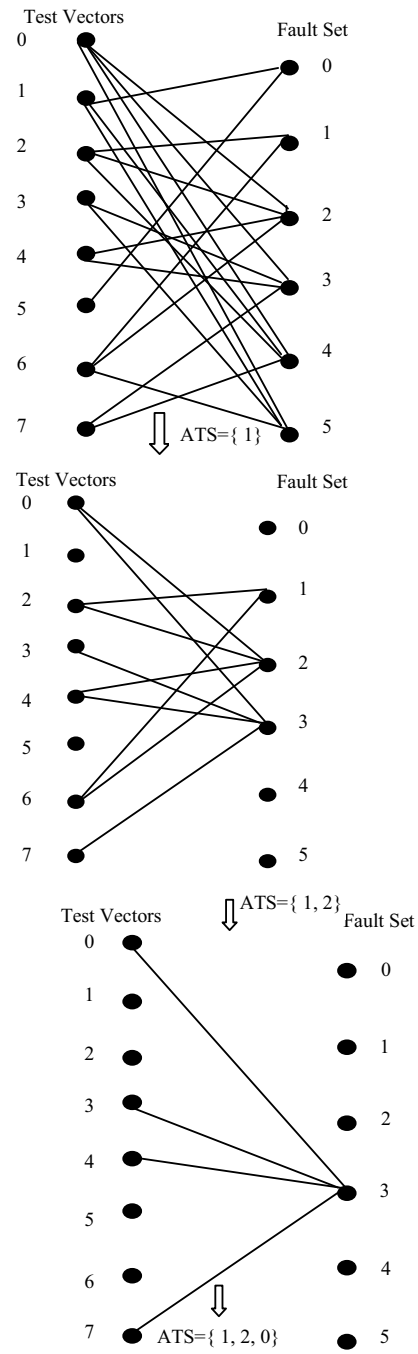

Fig. 7: Bipartite Graphs of ham3 benchmark circuit

In this case, the candidates are 0 and 1 which have degree 2. So, we can select any one of them. Here, the selected fault number is 0. Next, we find a set of test patterns which detect the fault 0. From this bipartite graph, we note that the set is {1, 5}. We then select a test vector from that set {1, 5} which has highest degree. Hence, we add this test vector 1 to ATS set. We delete all the edges of the test vector 1 and the fault nodes which are detected by this vector. We repeat the same process until all faults are covered. The ATS for PMGFs for the above circuit is {1, 2, 0}. The total number of SMGFs of the ham3

circuit is 5. After constructing the bipartite graph for SMGFs between the set of test patterns and set of possible SMGFs, we apply the same method to construct the test set. For ham3 circuit, the automatic test set for all SMGFs is {3, 2}. Total number of MMGFs of ham3 circuit is 5×(5+1)/2=15 and the automatic test set for all MMGFs is {3, 2}.

Table 2: Experimental of PMGFs

# Sl.No.	Circuit	*N*	*n*	Polian [22]	Proposed Method
				#PMGF	#PMGF
1	2of5d1	18	6	8	8
2	2of5d2	12	7	3	3
3	3 17tc	6	3	2	3
4	4 49tc1	16	4	5	6
5	5mod5tc	17	6	5	4
6	ham3tc	5	3	3	3
7	ham7tc	24	7	4	3
8	hwb4tc	17	4	5	5
9	mod5adders	21	6	6	7
10	mod5d1	8	5	2	2
11	mod5d2	9	5	2	2
12	rd32	4	4	3	3
13	rd53d1	12	7	8	8
14	rd53rcmg	30	7	8	7
15	xor5d1	4	5	1	1
Average				4.33	4.33

Table 3: Experimental of SMGFs

# Sl.No.	Circuit	*N*	*n*	Polian [22]	Proposed Method
				#SMGF	#SMGF
1	2of5d1	18	6	4	4
2	2of5d2	12	7	2	1
3	3 17tc	6	3	2	2
4	4 49tc1	16	4	3	3
5	5mod5tc	17	6	1	2
6	ham3tc	5	3	2	2
7	ham7tc	24	7	4	1
8	hwb4tc	17	4	2	2
9	mod5adders	21	6	3	3
10	mod5d1	8	5	1	1
11	mod5d2	9	5	1	1
12	rd32	4	4	2	2
13	rd53d1	12	7	2	1
14	rd53rcmg	30	7	3	1
15	xor5d1	4	5	1	1
Average				2.2	1.8

4. EXPERIMENTAL RESULTS

We have tested this procedure using several reversible benchmark circuits [13] as shown in Table 2. Column 2 shows the circuit name, column 3 and column 4 denote the gate count (*N*) and the input size (*n*) respectively. The number of tests for detecting all PMGFs/SMGFs/MMGFs obtained by running the ATPG [22], is shown in column 5 of Table 2/Table 3/Table 4 respectively. The number of tests for detecting all PMGFs/SMGFs/MMGFs obtained by the proposed algorithm is shown in column 6 of Table 2/Table 3/Table 4 respectively.

5. CONCLUSIONS

This paper presents an efficient method for deriving automatic test sets for detecting all PMGFs/SMGFs/MMGFs in a reversible circuit. Experiments show that the proposed technique produces improved results compared to the earlier methods.

Table 4: Experimental of MMGFs

# Sl.No.	Circuit	*N*	*n*	Polian [22]	Proposed Method
				#MMGF	#MMGF
1	2of5d1	18	6	5	5
2	2of5d2	12	7	2	1
3	3 17tc	6	3	2	2
4	4 49tc1	16	4	3	3
5	5mod5tc	17	6	6	2
6	ham3tc	5	3	2	2
7	ham7tc	24	7	4	1
8	hwb4tc	17	4	4	4
9	mod5adders	21	6	4	4
10	mod5d1	8	5	4	4
11	mod5d2	9	5	2	2
12	rd32	4	4	2	2
13	rd53d1	12	7	3	1
14	rd53rcmg	30	7	4	1
15	xor5d1	4	5	1	1
Average				3.2	2.3

REFERENCES

1. R. Landauer, "Irreversibility and heat generation in the computing process," *IBM Research and Development*, vol. 5, pp. 183-191, 1961.
2. R. P. Feynman, *Feynman Lectures on Computation.* (A. J. G. Hey and R. W. Allen, Ed.), Perseus Books, USA, 1996.
3. M. Nielsen and I. Chuang, *Quantum Computation and Quantum Information.* Cambridge University Press, 2000.
4. P. Picton, "Optoelectronic, multivalued, conservative logic," *International Journal of Optical Computing*, vol. 2, pp. 19-29, 1991.
5. R. C. Merkle, "Reversible electronic logic using switches," *Nanotechnology*, vol. 4, pp. 21-40, 1993.
6. R. Feynman, "Quantum mechanical computers," *Optics News*, vol. 11, pp. 11-20, 1985.
7. E. Fredkin and T. Toffoli, "Conservative logic," *Int. J. of Theoretical Physics*, vol. 21, pp. 219-253, 1982.
8. T. Toffoli, "Reversible computing," Tech Memo -MIT/LCS/TM-151, MIT Lab for Comp. Sci., 1980.
9. V. V. Shende, A. K. Prasad, I. L. Markov, and J. P. Hayes, "Synthesis of reversible logic circuits," *IEEE TCAD*, vol. 22, pp. 723-729, June 2003.
10. K. Iwama, Y. Kambayashi, and S. Yamashita, "Transformation rules for designing CNOT-based quantum circuits," *DAC* 2002, pp. 419-424.
11. D. M. Miller and G. W. Dueck, "Spectral techniques for reversible logic synthesis," *6th Intl. Symposium on Representations and Methodology of Future Computing Technologies*, pp. 56-62, March 2003.
12. D. Maslov, *Reversible Logic Synthesis.* Ph.D Thesis, University of New Brunswick, Canada, September 2003.
13. D. Maslov, G. Dueck, and N. Scott, *Reversible Logic Synthesis Benchmarks:* www.cs.uvic.ca/~dmaslov, 2004.
14. H. Rahaman, D. Kole, D. K. Das, and B. B. Bhattacharya, "On the detection of missing-gate faults in reversible circuits by a universal test set", *VLSI Design,* pp. 163-168, 2008.
15. K. N. Patel, J. P. Hayes, and I. L. Markov, "Fault testing for reversible circuits," *VTS*, pp. 410-416, 2003.
16. J. P. Hayes, I. Polian, B. Becker, "Testing for missing-gate faults in reversible circuits," *Asian Test Symposium*, pp. 100-105, 2004.
17. K. Ramasamy, R. Tagare, E. Perkins and M. Perkowski,"Fault localization in reversible circuits is easier than for classical circuits," *ATS 04*.
18. D. P. Vasudevan, P. K. Lala and J. P. Parkerson, "A novel approach for on-line testable reversible logic circuit design," *ATS 2004*, pp. 325-330.
19. A. Chakraborty, "Synthesis of reversible circuits for testing with universal test set and C-testability of reversible iterative logic arrays," *VLSI Design*, pp. 249-254, 2005.
20. H. Rahaman, D. K. Kole, D. K. Das, and B. B. Bhattacharya, "Detection of bridging faults in a reversible circuit", *VLSI Design and Test*, pp. 384-392, 2006.
21. H. Rahaman, D. K. Kole, D. K. Das, and B. B. Bhattacharya, "Minimal test set for bridging fault detection in reversible circuits", *ATS,* Bejing, China, pp. 125-128, 2007.
22. I. Polian, J. P. Hayes, T. Fienn, and B. Becker, "A family of logical fault models for reversible circuits", *ATS,* Kolkata, India, 2005, pp. 422-427.